

HomeSense – Problem-Solving & Design Explanation

GCIS-123 Assignment 1 · Task 9 Group No.: _____ Members: _____

1. Problem decomposition

We broke HomeSense into small functions that each do one job, then connected them in a main program. Each function returns its result instead of only printing it, so other functions and our pytest tests can use the value.

Function	Job	Returns
<code>operatingRange(device)</code>	Looks up the normal range of a device (1–5)	(min, max), or None for an unknown device
<code>severity(device, energy)</code>	Classifies one reading using that range	"Normal", "High" or "Critical"
<code>attention(device, energy)</code>	Decides whether the homeowner must act	True for High/Critical, False for Normal
<code>cost(energy, rate)</code>	Estimates the cost in AED	$\text{energy} \times \text{rate}$, or None for negative values
<code>feedback(device, energy)</code>	Chooses a recommendation	One of three messages
<code>highestCons(device)</code>	Converts a device number into its name for the report	Device name
<code>main()</code>	Loops over the readings, calls the functions, keeps totals, prints the report	–

The functions build on each other: `operatingRange` → `severity` → `attention` and `feedback`. The classification rule is written in one place only (`severity`). The `attention` and `feedback` functions call `severity` instead of repeating the rule, so if the rule changes, the whole program stays consistent.

2. Decision making: Normal, High and Critical

The reference table already defines what is Normal, so we used it directly. For readings above the normal range, we measured how far the reading is above the device's own upper limit (max):

- **Normal** – $\text{energy} \leq \text{max}$
- **High** – above max, up to $1.5 \times \text{max}$ (up to 50% above the normal limit)
- **Critical** – above $1.5 \times \text{max}$ (more than 50% above the normal limit)

Why this rule is reasonable:

- A small overshoot usually has an ordinary cause, such as a device left on longer or a low thermostat setting. The homeowner should check it, but it is not an emergency, so it is High.
- A reading more than half again above the normal maximum is unlikely to be normal use and may mean a fault. It is Critical, and the feedback tells the homeowner to inspect or disconnect the device.
- The rule agrees with the assignment's own example: an Air Conditioner at 7.5 kWh (exactly 1.5×5.0) is classified as High.

- We chose 50% as a middle point. A smaller margin (for example 10%) would call ordinary variation Critical. A larger one (for example 100%, double the maximum) would leave a clearly faulty device at only High.

3. Boundary cases

A reading exactly equal to the upper normal limit is **Normal**. We compare with $\leq$ (energy $\leq$ max) because the ranges in the table include their end values: “0.01 – 0.10 kWh” means 0.10 is still normal. For example, LED Light at 0.10 kWh is Normal, while 0.11 kWh is High. The same applies at the High/Critical limit: exactly $1.5 \times$ max is still High, and anything above it is Critical.

Case	Example	Result
At the lower normal limit	LED Light, 0.01 kWh	Normal
At the upper normal limit	LED Light, 0.10 kWh	Normal
Just above the upper limit	LED Light, 0.11 kWh	High
Exactly $1.5 \times$ max	Air Conditioner, 7.50 kWh	High
Above $1.5 \times$ max	LED Light, 0.20 kWh	Critical
Zero (below the minimum)	LED Light, 0 kWh	Normal: low use wastes no energy; the device is off or idle
Negative energy	LED Light, -1 kWh	Rejected as invalid before any calculation
Unknown device	Device 6	Error message; the reading is not counted

4. Different devices

Devices naturally use very different amounts of energy. The Air Conditioner's normal maximum (5.00 kWh) is 50 times the LED Light's (0.10 kWh), so one threshold cannot work for both:

- If the threshold were set for the LED Light (0.10 kWh), every normal Air Conditioner reading would be flagged, and the homeowner would get constant false alarms.
- If it were set for the Air Conditioner (5.00 kWh), an LED Light using 1 kWh (ten times its normal maximum, which is clearly a fault) would still look Normal.

So each device has its own range, and our High and Critical limits are calculated from that device's own maximum. The rule is the same for every device, but the numbers change with the device (see the table in the classification rule statement below).

5. Multiple readings

The single-reading functions did not change. Moving to many readings changed the main program:

- **Loop:** one for loop runs the same steps for every reading, instead of separate code for each of the 10 readings.
- **Accumulators:** before the loop we set the counters and totals to zero (readings, Normal, High, Critical, attention, total energy, total cost). Each pass through the loop adds to them.

- **Running maximum:** we keep the highest reading so far and its device name, and replace them whenever a larger reading appears.
- **Validation inside the loop:** an unknown device or a negative value is skipped with continue, so one bad reading does not crash the program or corrupt the totals.
- **Report after the loop:** the summary is printed once, after all readings are processed, using the totals.

For the 10 sample readings the program reports 5 Normal, 2 High and 3 Critical readings, 5 requiring attention, a total of 20.96 kWh, an estimated cost of AED 6.29 (at 0.30 AED/kWh), and the highest consumption as Air Conditioner – 7.50 kWh. The report in the assignment brief only shows the format; the actual sample readings add up to 20.96 kWh.

6. Testing

The most important tests were the ones at the edges of our rule. A wrong comparison (for example < instead of <=) only shows up at a boundary, while a reading in the middle of a range passes almost any rule.

- **Upper limit and just above** (LED Light 0.10 → Normal, 0.11 → High): confirm the Normal/High boundary is correct.
- **High/Critical limit** (Air Conditioner 7.50 → High, 7.51 → Critical): confirm our own $1.5 \times \text{max}$ rule.
- **Negative energy** (–1 kWh): during boundary testing this crashed the program, because cost() returned None and it could not be added to the total. We changed the main program to reject negative values before any calculation.
- **Unknown device** (device 6): the program first went on to ask for the energy value anyway. Adding continue made it skip the reading.
- **Cost calculation** ($4.0 \text{ kWh} \times 0.30 \text{ AED/kWh} = 1.20 \text{ AED}$): confirms the arithmetic behind the report totals.

Our pytest tests use assert to check the value each function returns, not just that the program runs without errors.

HomeSense High/Critical classification rule

For a device whose normal range is min – max (from the reference table):

- **Normal** – energy $\leq \text{max}$. Readings below min, including 0 kWh, are also Normal.
- **High** – $\text{max} < \text{energy} \leq 1.5 \times \text{max}$
- **Critical** – energy $> 1.5 \times \text{max}$
- Negative energy is invalid and is rejected before classification.

Device	Normal (kWh)	High (kWh)	Critical (kWh)
LED Light	up to 0.10	above 0.10 to 0.15	above 0.15
Television	up to 0.50	above 0.50 to 0.75	above 0.75
Refrigerator	up to 1.50	above 1.50 to 2.25	above 2.25
Washing Machine	up to 2.50	above 2.50 to 3.75	above 3.75
Air Conditioner	up to 5.00	above 5.00 to 7.50	above 7.50